Universidad Nacional de Costa Rica Sede Regional Brunca ·
Campus Coto y Pérez Zeledón

Informe del Proyecto II

Rutas aéreas: grafos, recorridos y ruta más corta

EIF207 Estructuras de Datos · II Ciclo 2026 · Entrega y defensa: viernes 06/11/2026

Integrantes: Allan Estrada Quesada 604940962

Repositorio: https://github.com/allanestradaq/proyecto2-EstructuraDeDatos.git

1. Descripción y arquitectura

El proyecto es un planificador de vuelos entre aeropuertos de América. Cada aeropuerto, identificado por su código IATA, es un **vértice** de un grafo dirigido y ponderado; cada ruta es una **arista** con dos pesos: duración en minutos y costo en dólares. Se desarrolló en Java con interfaz gráfica Swing. BFS, DFS y Dijkstra se escribieron a mano con las colecciones estándar de `java.util`; no se usó ninguna biblioteca de grafos y ningún dato está escrito en el código: todo se lee de `aeropuertos.txt` y `rutas.txt`.

Paquete	Responsabilidad
`modelo`	Datos inmutables: `Aeropuerto`, `Ruta`, `Itinerario` y `Criterio` (qué peso usa cada consulta).
`grafo`	`GrafoAereo`: lista de adyacencia, cancelar y restaurar rutas, grafo invertido.
`carga`	`CargadorDatos`: lectura de los dos archivos con `Scanner` y reporte de líneas inválidas.
`algoritmos`	`Bfs`, `Dijkstra` y `Dfs`. No dependen de la interfaz.
`servicio`	`PlanificadorVuelos`: valida los códigos y elige el algoritmo según el criterio; maneja los casos especiales.
`vista`	`VentanaPrincipal`, `PanelMapa` (dibujo) y `AnimadorVuelo` (tiempo de la animación).
`app`	`Main` (interfaz) y `Analisis` (consola, para obtener los datos de este informe).

La separación en capas permite probar los algoritmos por consola, y cambiar la interfaz sin tocar la lógica.

2. Estructuras de datos y justificación

Estructura	Dónde se usa	Por qué
`LinkedHashMap<String, Aeropuerto>`	Catálogo de aeropuertos	Acceso por código en O(1) promedio y orden de inserción estable (resultados reproducibles).
`LinkedHashMap<String, List<Ruta>>` con `ArrayList`	Lista de adyacencia	Grafo disperso: los vecinos de un vértice se recorren en O(grado) y una ruta se quita sin reorganizar nada más.
`ArrayList<Ruta>`	Rutas canceladas	Permite restaurarlas sin releer el archivo, que nunca se modifica.
`ArrayDeque` como cola	BFS	FIFO en O(1): produce el recorrido por niveles, por lo que el destino se descubre con la menor cantidad de vuelos.
`ArrayDeque` como pila	DFS	LIFO en O(1). Evita la recursión, que podría desbordar la pila de llamadas en redes de miles de aeropuertos.
`PriorityQueue` y `HashMap` de distancias	Dijkstra	Extrae el vértice de menor distancia en O(log n) en lugar de recorrer todos.
`HashSet` / `LinkedHashSet`	Visitados y alcanzables	Pertenencia en O(1); el segundo conserva el orden de visita.

`HashMap<String, Ruta>` (previo)	BFS y Dijkstra	Guarda la arista con la que se llegó a cada vértice y permite reconstruir el camino hacia atrás.
`record` (Java 17)	Aeropuerto, Ruta, Itinerario	Datos inmutables sin código repetitivo; ningún algoritmo puede alterarlos por accidente.

Un solo Dijkstra. «Más rápida» y «más barata» usan la misma implementación: el peso de cada arista llega como parámetro (`ToDoubleFunction<Ruta>`), que vale `Ruta::minutos` o `Ruta::costo` según el criterio. Ese parámetro es lo único que cambia.

3. Complejidad de cada operación

Sea *n* la cantidad de aeropuertos, *m* la de rutas, *g* el grado de salida de un aeropuerto y *c* la cantidad de rutas canceladas.

Operación	Complejidad	Justificación
Cargar los dos archivos	O(n + m)	Se lee cada registro una vez; cada inserción en los mapas es O(1) promedio.
Consultar un aeropuerto y sus rutas	O(1 + g)	Búsqueda por código en O(1) y recorrido de su lista de salientes.
Agregar una ruta	O(1) amortizado	Se agrega al final de la lista del origen.
Cancelar una ruta	O(g)	Se recorre solo la lista de salientes del origen.
Restaurar las rutas canceladas	O(c)	Se reinserta cada ruta cancelada.
BFS (menos escalas)	O(n + m)	Cada vértice entra una vez a la cola y cada arista se examina una vez.
DFS (alcanzables)	O(n + m)	Cada vértice se visita una vez y cada arista se examina una vez.
Dijkstra (más rápida o más barata)	O((n + m) log n)	Hasta m inserciones en la cola; cada `add` o `poll` cuesta O(log n). La cola tiene a lo sumo m entradas y log m ≤ 2 log n.
Construir el grafo invertido	O(n + m)	Se copian los vértices y se invierte cada arista.
Rutas críticas desde un origen (pregunta 4)	O(m · (n + m))	Un DFS por cada ruta que se simula cancelar.

4. Preguntas de análisis

1. Rutas de DEN → SJO, LIR → SJU y SJO → MVD

Viaje	Criterio	Ruta	Vuelos	Minutos	Costo ($)
DEN → SJO	Menos escalas (BFS)	DEN -> LAX -> SJO	2	500	895
	Más rápida (Dijkstra)	DEN -> DFW -> IAH -> SJO	3	385	525
	Más barata (Dijkstra)	DEN -> DFW -> SAL -> MGA -> SJO	4	425	435
LIR → SJU	Menos escalas (BFS)	LIR -> SAL -> MIA -> SJU	3	385	405
	Más rápida (Dijkstra)	LIR -> SJO -> PTY -> CCS -> SJU	4	355	520
	Más barata (Dijkstra)	LIR -> SAL -> MIA -> SJU	3	385	405
SJO → MVD	Menos escalas (BFS)	SJO -> MIA -> BSB -> GRU -> MVD	4	870	1125
	Más rápida (Dijkstra)	SJO -> PTY -> LIM -> EZE -> MVD	4	595	685

	Más barata (Dijkstra)	SJO -> PTY -> LIM -> GRU -> MVD	4	720	640

¿Por qué la ruta de menos escalas no es la más rápida? Porque BFS cuenta vuelos, no minutos: trata todas las aristas como iguales. Un vuelo directo puede durar más que dos vuelos cortos encadenados. En SJO -> MVD la ruta con menos escalas tiene 4 vuelos y 870 minutos, mientras que la más rápida tiene 4 vuelos y 595 minutos.

¿Por qué la más rápida no es la más barata? Porque minutos y dólares son dos pesos independientes: la ruta que minimiza uno no tiene por qué minimizar el otro. En SJO -> MVD la más rápida cuesta $685 y la más barata, $640, pero tarda 720 minutos.

2. BFS, condición de Dijkstra y costos negativos

BFS no sirve para la ruta más rápida porque minimiza la cantidad de aristas y nunca consulta los minutos. Solo coincidiría con la más rápida si todos los vuelos duraran lo mismo.

Condición de Dijkstra: todos los pesos deben ser no negativos. Así, cuando un vértice sale de la cola su distancia ya es definitiva: cualquier otro camino hacia él pasa por vértices con distancia mayor o igual, y sumar pesos no negativos no puede mejorarla. Esto se cumple en los datos entregados (minutos y dólares ≥ 0).

Con un costo negativo (un reembolso) esa garantía se rompe. Ejemplo: $A \to B$ cuesta 4, $A \to C$ cuesta 5 y $C \to B$ cuesta −3. El camino $A \to C \to B$ cuesta 2, pero Dijkstra extrae B con distancia 4 antes de procesar C y devuelve 4, que es incorrecto. Si además existiera un **ciclo de costo negativo**, no habría mínimo, porque se podría dar vueltas reduciendo el costo sin límite. Para pesos negativos se usa Bellman-Ford. Nuestra implementación lanza `IllegalArgumentException` si encuentra un peso negativo.

3. Aeropuertos no alcanzables desde SJO y aeropuertos desde los que no se llega a SJO

- **No alcanzables desde SJO:** KIN
- **Desde los que no es posible llegar a SJO:** ASU, MVD

Cómo lo calcula el programa. Para los primeros se ejecuta DFS desde SJO; los no alcanzables son todos los aeropuertos menos los visitados, en O(n + m). Para los segundos se construye el **grafo invertido** (todas las rutas al revés) y se ejecuta DFS desde SJO: los visitados son exactamente los que pueden llegar a SJO. Es mucho más barato que ejecutar un DFS desde cada aeropuerto: O(n + m) frente a O(n · (n + m)).

4. Ruta cuya cancelación deja un aeropuerto inalcanzable desde SJO

- **Ruta encontrada:** SJO -> KIN· **Aeropuerto que queda inalcanzable:** KIN
- **¿Es la única?** no

Cómo se encontró. Para cada una de las *m* rutas se simula su cancelación (el DFS la ignora con un filtro, sin modificar el grafo), se recalculan los alcanzables desde SJO y se comparan con los originales. Si algún aeropuerto desaparece, la ruta es crítica. El costo es O(m · (n + m)). Como atajo de razonamiento: si un aeropuerto alcanzable tiene una sola ruta entrante, esa ruta es crítica; la condición es suficiente pero no necesaria, porque una ruta también puede ser el único acceso a todo un grupo de aeropuertos. El resultado se comprobó en la aplicación: se canceló la ruta y la consulta «Alcanzables» desde SJO mostró 1 en gris.

5. Lista de adyacencia frente a matriz de adyacencia

	n = 40, m = 202	**n = 4 000, unas 20 rutas por aeropuerto (m ≈ 80 000)**
Matriz (2 pesos, `int` de 4 bytes)	$40^2 \times 2 \times 4 = 12\,800$ B ≈ 12,5 KB	$4\,000^2 \times 2 \times 4 = 128\,000\,000$ B ≈ 122 MB
Lista de adyacencia	242 elementos (n + m), del orden de 10 KB	84 000 elementos, del orden de 4 MB
Celdas ocupadas de la matriz	202 / 1 600 ≈ 12,6 %	80 000 / 16 000 000 ≈ 0,5 %

Los bytes de la lista son un orden de magnitud: un objeto `Ruta` (dos referencias y dos `double`) ocupa unos 40 bytes en una JVM de 64 bits, más su referencia en la lista. Con `double` en lugar de `int`, la matriz se duplica.

Con 40 aeropuertos la diferencia es pequeña y cualquiera de las dos cabe sin problema. Con 4 000, la matriz crece con n^2 (decenas de veces más memoria) y casi toda está vacía, mientras que la lista crece con $n + m$. La lista también es más rápida donde importa: recorrer los vecinos de un vértice cuesta O(grado) y no O(n), por lo que BFS y DFS son O($n + m$) y no O(n^2). La matriz solo gana al consultar si existe una ruta concreta A → B (O(1) frente a O(grado)) y cuando el grafo es denso.

6. Complejidad de BFS, DFS y Dijkstra

Algoritmo	Complejidad	Por qué
BFS	O(n + m)	Cada vértice entra una vez a la cola y cada arista se examina una vez.
DFS	O(n + m)	Cada vértice se visita una vez y cada arista se examina una vez.
Dijkstra con `PriorityQueue`	O((n + m) log n)	Hasta m inserciones; cada `add` o `poll` cuesta O(log n).
Dijkstra buscando el mínimo recorriendo todos los vértices	O(n² + m)	n extracciones, y cada una recorre hasta n vértices.

Qué cambia con la cola de prioridad. El montículo entrega el mínimo en O(log *n*) en lugar de O(*n*). En grafos dispersos gana por mucho: con $n = 4\,000$ y $m = 80\,000$, la versión lineal hace del orden de $n^2 = 16\,000\,000$ pasos y la versión con cola unos $84\,000 \times \log 4\,000 \approx 1\,000\,000$. En grafos muy densos ($m$ cercano a n^2) la versión lineal, $O(n^2)$, supera a la 2 2 del montículo, O(*n* log *n*). Como el enunciado trabaja con redes de pocas rutas por aeropuerto, la cola de prioridad es la elección adecuada.

5. Pruebas y limitaciones

Pruebas realizadas. Se probo el alcance de cada uno de los nodos

- Trazas a mano de BFS, DFS y Dijkstra sobre un ejemplo de 5 aeropuertos y 6 rutas; coinciden con la salida del programa.
- Los archivos entregados (40 aeropuertos, 202 rutas) y archivos propios del mismo formato: un aeropuerto aislado, uno al que nadie llega, una ruta con código inexistente (se informa como advertencia) y un archivo con el contador de líneas incorrecto (se muestra un error claro).
- Casos especiales: código inexistente, origen igual al destino (0 vuelos) y destino sin ruta («No existe ruta de X a Y»).
- Cancelar una ruta, repetir la consulta y restaurar todas: los resultados cambian y vuelven a los originales.

Limitaciones. Dijkstra rechaza pesos negativos, por lo que no modela reembolsos. La ruta a cancelar se elige con los campos de origen y destino. El mapa supone la ventana de 1000 × 1000 de los archivos y la encaja en el tamaño del panel. Los minutos y costos son ficticios.

Herramientas de apoyo. Se utilizo ia para modificar el paquete de vistas, y llevar un listado de pendientes